The data is pretty much the same except for how it is formed, formatted, and stored in these structures: database structures are relational, while those in Elasticsearch are denormalized and nonrelational. A document in Elasticsearch consists of self-contained information with no relations.

As you can see on the right side in figure 3.4, the `STUDENT` and `ADDRESS` tables are related by the foreign key `ADDRESS_ID`, which appears in both tables. Elasticsearch doesn't have the concept of relationships (or, shall we say, a limited scope of relationships is supported), so the entire student document is stored in a single index. The inner objects (`address` field in figure 3.4) are in the same index as the main fields.

> **NOTE** Data in Elasticsearch is denormalized to assist with speedy search and retrieval, unlike in a relational database, where data is normalized in various forms. While you can create parent-child relationships to some extent in Elasticsearch, doing so can lead to bottlenecks and performance degradation. If your data is expected to be relational, Elasticsearch may not be the right solution.

Just as we can insert multiple records in a table, we can index several JSON documents into Elasticsearch. However, unlike in a relational database, where we must create the table schema up front, Elasticsearch (like other NoSQL databases) lets you insert documents without a predefined schema. This *schema-less* feature comes in handy during testing and development but can also be a problem in production environments.

Document operation APIs

As a standard across Elasticsearch, indexing documents is expected to use clearly defined document APIs. There are two types of document APIs: those that work on a single document and those that work on multiple documents at once (batches). We can index or retrieve documents one by one using single-document APIs or batch them up using multidocument APIs (exposed as RESTful APIs over HTTP). These are briefly described here:

- *Single-document APIs*—Perform actions on individual documents, one by one. More like CRUD-related operations (create, read, update, and delete), these APIs let us get, index, delete, and update documents.
- *Multiple-document APIs*—Work with multiple documents at once. They allow us to delete and update multiple documents with a single query, index in bulk, and reindex data from source index to target index.

Each API has a specific usage, as we discuss in detail in chapter 5.

While we are on the subject of documents, one thing often comes up: document types. Although document types are deprecated and removed from version 8.x, they may appear on your radar when working with Elasticsearch and confuse you, especially with older (version 5.x or less) versions of Elasticsearch. So, let's take a moment to understand what they are and their current state before learning about indexes.